Decisiones técnicas (consigna 4.7)

Criterios usados en todas las decisiones: **costo** (todo debía ser gratuito), **curva de aprendizaje** para dos integrantes, **calidad de la documentación** y **adecuación al problema** (pocas pantallas, reglas de cupo y horario que tienen que cumplirse aun con pedidos simultáneos, y cobro en pesos).

1. Frontend — React 18 + Vite (TypeScript), sin librerías de UI ni router. *Alternativas:* Angular; Vue 3; Next.js. *Fundamento:* React tiene la documentación y la cantidad de ejemplos más grandes y ya lo conocíamos; Vite construye un sitio estático que se publica gratis en un CDN y arranca aunque la API no esté (consigna 3.1). Angular exige aprender mucho más (módulos, RxJS) para cuatro pantallas; Next.js agrega renderizado en el servidor, que no necesitamos y complica el despliegue estático. *Se resigna:* las herramientas que trae Angular "de fábrica" (formularios, router) y el SEO del renderizado en servidor; las pestañas se manejan con estado.

2. Backend — Node 22 + Express 5 + TypeScript, validación con zod. *Alternativas:* Python con FastAPI; Java con Spring Boot. *Fundamento:* el mismo lenguaje que el frontend (los tipos se entienden de los dos lados); Express es mínimo y muy documentado; Express 5 lleva solos los errores de las rutas `async` al manejador central; zod da errores 400 claros por campo. Spring Boot es más pesado de aprender y de correr en un plan gratuito de 512 MB. *Se resigna:* la estructura impuesta de un framework más grande (la organización por módulos la definimos nosotros) y el tipado de Python con FastAPI, que genera la documentación de la API sola.

3. Base de datos — PostgreSQL en Neon (plan gratuito), con Prisma como ORM. *Alternativas:* MySQL en Aiven o Railway; MongoDB Atlas; Supabase. *Fundamento:* el negocio es relacional (usuarios, menús, pedidos y pagos con claves foráneas) y la regla de cupo necesita **transacciones con bloqueo de fila** (`SELECT … FOR UPDATE`), que PostgreSQL resuelve bien; MongoDB complicaría esa garantía. Neon es gratis, remoto y con TLS. Prisma da migraciones versionadas en el repositorio y consultas tipadas. *Se resigna:* el panel y la autenticación incluidos de Supabase, y algo de control fino del SQL (los bloqueos se escriben con `$queryRaw`). El plan gratuito tiene poco almacenamiento y la base se suspende sin uso (la primera consulta tarda un poco más).

4. Autenticación — propia, con bcrypt y JWT (8 h) y roles en la tabla `Rol`. *Alternativas:* Auth0 o Clerk (proveedor externo); sesiones con cookie guardadas en la base. *Fundamento:* son pocas reglas, se pueden probar con tests de integración (401 y 403) y se explican completas en la defensa. El JWT no necesita guardar sesiones; para poder cerrar sesión de verdad se agregó `versionToken`, que invalida los tokens viejos. *Se resigna:* inicio de sesión con Google y verificación del email que darían los proveedores externos; como el token vive en `localStorage`, depende de que el frontend no tenga XSS (React escapa el contenido).

5. Despliegue y CI/CD — Render (API y sitio estático) + GitHub Actions. *Alternativas:* Vercel (frontend) + Railway (API); un VPS con Docker. *Fundamento:* Render tiene plan gratuito para los dos componentes, HTTPS automático, variables de entorno como secretos y *deploy hooks*; GitHub Actions es gratis en el repositorio y permite que el despliegue dependa de que los tests y la cobertura (≥ 65 %) pasen. Railway ya no tiene plan gratuito permanente; un VPS cuesta dinero y exige administrar el servidor. *Se resigna:* la API se duerme tras 15 minutos sin uso (la primera petición tarda cerca de un minuto) y los logs gratuitos de Render guardan poco historial.

6. Tecnología avanzada — integración con Mercado Pago (Checkout Pro), más logs estructurados. *Alternativas:* observabilidad sola con una herramienta externa (Better Stack, Grafana Cloud); caché con Redis. *Fundamento:* resuelve un problema real del negocio: las reservas que no se pagan ocupan cupo; ahora una reserva sólo se confirma con el pago aprobado y se libera sola si vence. El estado se consulta siempre a Mercado Pago (no se confía en el navegador) y la falla o demora del servicio responde 502/503 sin dejar reservas trabadas. Mercado Pago es el medio de pago más usado en Argentina, tiene cuentas de prueba gratuitas y se usó con `fetch` sin SDK. Una caché no tiene sentido con tan pocos datos. Los logs JSON (pino) en Render permiten ver qué pasó con un pedido en un momento dado. *Se resigna:* Mercado Pago cobra comisión por venta en producción; en local no llega el webhook (se confirma cuando el cliente vuelve), y sin una herramienta de logs externa el historial es corto.

Declaración de uso de inteligencia artificial (consigna 8)

Durante el desarrollo se usó **Claude (Anthropic)**, a través de Claude Code, para: generación de código del backend y del frontend, escritura de tests, interpretación de errores del pipeline y de Mercado Pago, redacción de la documentación y elaboración de los diagramas (DFD, casos de uso, DER y arquitectura, a partir del código del repositorio). Los integrantes definieron el producto y sus reglas de negocio (ver la propuesta), eligieron entre las alternativas de esta página, probaron el sistema funcionando y asumen la responsabilidad de comprender y explicar todo lo entregado.